Why you should work on btcd

Full node implementation in Go





Water deer. My spirit animal

github.com/kcalvinalvin



Seoul Bitcoin Meetup since 2017

https://www.meetup.com/seoulbitcoin/



github.com/btcsuite/btcd



Main contributions as a developer

https://eprint.iacr.org/2019/611

100%

btcd is easier to read than any other implementation

"btcd code is so much better than Bitcoin Core's"

roasbeef - CTO of Lightning Labs

Names explain the code

Bitcoin Core

```
int nSize = nCompact >> 24;
uint32_t nWord = nCompact & 0x007fffff;
```

rust-bitcoin

```
let unshifted_expt = bits >> 24;
let (mant, expt) = { /* ... */ };
```

libbitcoin
parse fields

```
bool negative;
exponent_type exponent;
small_type mantissa;
```

btcd

```
mantissa := compact & 0x007fffff
isNegative := compact&0x00800000 != 0
exponent := uint(compact >> 24)
```

Why include the input amount?

BIP143 signature hashing

Bitcoin Core

```
// Witness sighashes need
// the amount.
```

rust-bitcoin

```
/// Computes the BIP-0143 sighash
/// to spend a p2wpkh transaction
/// for any flag type.
```

libbitcoin

```
// NOT THREAD SAFE
// Concurrent input validation for a tx
// unsafe due to on-demand hash caching.
```

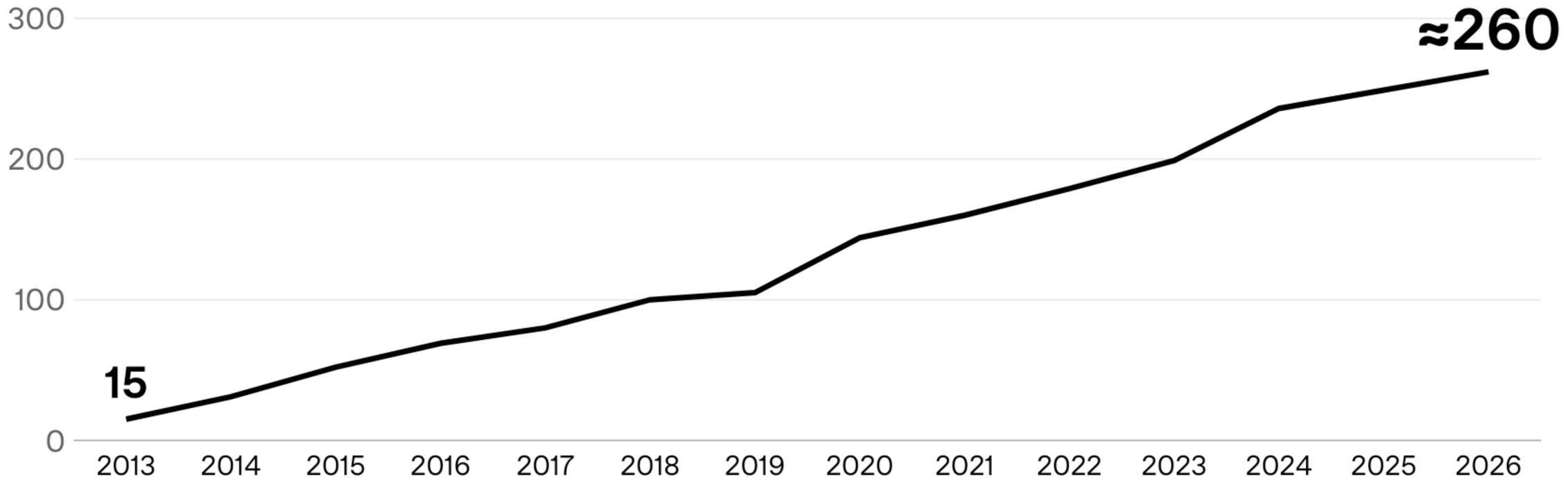
btcd

```
// This allows offline, or hardware
// wallets to compute the exact amount
// being spent, in addition to the
// final transaction fee.
//
// [...] invalid input amount, the real
// sighash will differ causing
// the produced signature to be invalid.
```

Selected source comments, rewrapped

btcd contributors

Cumulative commit authors



btcsuite/btcd git history, as of 11 Nov 2026. Aliases combined; counts approximate.



